COMPUTER SCIENCE + COMPUTER ENGINEERING

06

Answer Key and Review Guide

Lesson quizzes, application checks and module reviews

100 lesson answers · 25 module-review answers · Readiness explanations

PACKAGE 06 / EXPANDED VOLUME 05 / SEPTEMBER 28, 2026
Self-study: derive, implement, test and explain.

Answer Key and Review Guide

Use after an independent attempt

Each lesson has five answers. Questions 1-3 retain the original roadmap questions; answers are expanded here. Questions 4-5 are new applications. Equivalent answers can pass when assumptions and reasoning are correct. Numerical answers without units/domain, or terminology without an implementation explanation, are incomplete.

Use the reference implementation to diagnose differences after forming your own hypothesis. Do not replace your starter with a copied file and treat a green test run as evidence of independent understanding. The 25 module-review answers are at the end.

Readiness check answers (separate from lesson totals)

1. x_ndc = 2/4 = 0.5, provided w is valid for the division.
2. 320*1804* = 230,400 bytes, excluding metadata and padding.
3. A non-owning pointer is only an address; copying it does not change ownership or the target lifetime.
4. 0.2+0.3+0.5 = 1.
5. Two full steps consume 20 ms. Five ms remains, giving interpolation fraction 0.5.
6. No. Submission/CPU return does not establish GPU completion or reusable ownership.
7. No. Normal vectors are data; apply their documented decode and normalization, not a color transfer by default.
8. Reject the candidate and preserve the current valid world; publish only a completely validated replacement.

Lesson 21.1 — Images, color and coordinate conventions

1. Why must image stride be recorded?

Stride specifies the actual byte distance between rows; padding makes width times pixel size insufficient.

2. How many bytes are in the tightly packed example image?

320*1804* = 230,400 bytes of tightly packed RGBA8 pixel storage, excluding header or metadata.

3. Should a normal map automatically receive sRGB decoding?

No. It stores directional data; a color transfer function would distort the encoded vector.

4. A 3-by-2 RGB8 image has stride 12. What is the byte offset of pixel (2,1), and how large is its allocation?

Offset 18, allocation 24 bytes. Only 18 bytes represent visible RGB samples.

5. Why does averaging encoded black and white produce a different result from encoding their linear-light average?

The transfer function is nonlinear. Average linear values first: 0.5 encodes to about 0.7354, not 0.5.

Diagnostic guidance: Upside-down output usually indicates an origin mismatch. Blue/red swaps indicate channel order. A too-dark gradient often indicates missing output encoding; a washed-out image can indicate double encoding. First inspect stored values, then inspect the viewer.

Lesson 21.2 — Projection and clipping before perspective division

1. Why clip before perspective division?

Clipping retains valid visible portions while homogeneous coordinates are well behaved; premature division loses the appropriate boundary relationship and can divide by zero or reverse signs.

2. What normalized component follows from x=2 and w=4?

0.5.

3. Is deleting every outside vertex a valid triangle-clipping algorithm?

No. A triangle can intersect the view volume even when some or all original vertices are outside; create edge intersections and retain the interior polygon.

4. For plane f=z, endpoints have distances -2 and 6. What interpolation parameter reaches the boundary?

t=-2/(-2-6)=0.25 from the first endpoint.

5. In a zero-to-one clip volume, does z=-0.2,w=1 pass the near plane? What changes under a minus-one-to-one depth convention?

It fails z>=0. With a matching minus-one-to-one projection and near rule z>=-w, it would pass that near-plane inequality. Other planes still need testing.

Diagnostic guidance: Exploding triangles suggest division happened too soon. Missing near-plane triangles suggest whole-triangle rejection replaced clipping. Seams suggest inconsistent intersection or fill rules. A left/right mirror suggests matrix or viewport conventions, not a need to reverse all normals.

Lesson 21.3 — Triangle coverage, depth and perspective-correct interpolation

1. What is the sum of barycentric coordinates?

They sum to 1 for the affine coordinate representation of a point in the triangle plane; numerical implementations have rounding error.

2. When does perspective-correct interpolation reduce to affine interpolation?

For arbitrary attributes, when all three w values are equal. Constant attributes also agree as a special case.

3. Why specify a shared-edge coverage rule?

It gives a shared boundary exactly one owner, preventing cracks or duplicate fragment processing under the chosen sample convention.

4. Compute perspective-correct u for weights (0.2,0.3,0.5), w=(1,2,4), u=(0,1,1).

Numerator 0.275, denominator 0.475, so u=11/19, approximately 0.578947.

5. Why is affine interpolation of z_ndc not the same mistake as affine interpolation of UVs?

Depth has already undergone the projection division; conventional raster depth is affine in screen barycentrics. A surface UV remains an attribute needing the reciprocal-w ratio.

Diagnostic guidance: Bending checkerboards usually indicate affine UVs. Random holes often indicate winding or edge-equality mistakes. Correct colors with wrong overlap suggest depth-space confusion. Do not loosen a global epsilon to conceal all three.

Lesson 21.4 — Rays, intersections and acceleration structures

1. When is ray parameter t directly a distance?

When d has unit length in the coordinate system used for distance; for forward rays, traveled distance equals nonnegative t.

2. What point occurs in the worked example at t=3?

(0,0,6).

3. Why retain the brute-force implementation?

It provides a simpler correctness oracle and baseline cost against which acceleration can be checked.

4. A sphere hit occurs at t=4 with unit direction. What t reaches the same point when the direction is doubled?

t=2. The product t*d must stay unchanged.

5. Why can a BVH with heavily overlapping node boxes approach brute-force behavior?

A ray intersects many overlapping bounds, so traversal cannot reject most subtrees and must test many primitives.

Diagnostic guidance: False misses often come from nonconservative bounds or an incorrect parallel-axis case. Wrong nearest hits often come from returning the first visited leaf instead of the nearest candidate. Acceleration is secondary until the oracle comparison passes.

Lesson 22.1 — GPU work submission, resources and pipeline state

1. Does queue submission mean the GPU has finished?

No. It schedules work; completion requires the API-specific completion evidence.

2. Why can immediate buffer destruction be unsafe?

Submitted or recorded work may still reference the allocation. Retire it only after all relevant users have finished.

3. Must a beginner implement three graphics APIs simultaneously?

No. Choose one compatible backend and learn its lifetime and synchronization rules.

4. A submit call returns after 0.2 ms. Can you conclude the GPU rendered in 0.2 ms? Explain.

No. That is CPU-side call duration, not a measured GPU execution interval. Use supported GPU timestamps or clearly labeled completion-latency observations.

5. A requested optional feature is absent. Give two valid policy responses and one invalid response.

Valid: disable the optional path and use a supported fallback, or reject startup with an actionable explanation when the feature is required. Invalid: use the feature anyway and claim support.

Diagnostic guidance: A successful device handle with no image does not prove that the render pass executed. Check target format, attachment usage, command submission and error scopes in order. Avoid disabling validation to hide a warning.

Lesson 22.2 — Buffers, shaders, layouts and resource binding

1. Why can matching field names still produce a layout bug?

The host and shader may disagree about alignment, padding, array stride, matrix order or scalar representation even when member names match.

2. How many 64-thread groups cover 1,000 elements?

16 groups; 1,024 invocations, of which 24 must not access an element.

3. What does a resource binding establish?

It connects a shader-visible location to a specific resource/range/view under the pipeline interface and usage rules.

4. A WGSL storage struct contains vec3 position and f32 scale. What are the member offsets and total size?

position at 0, scale at 12, total 16 for this layout. This does not imply a 12-byte array stride for vec3.

5. How many 64-thread groups cover 1,025 elements, and why is a bounds guard still required?

17 groups create 1,088 invocations; 63 lie beyond 1,025 and must not access the array.

Diagnostic guidance: Exploded geometry often comes from stride or matrix interpretation. Correct first elements with corrupted later elements often indicate array stride. A failure only at N=65 often indicates a missing bounds guard.

Lesson 22.3 — Synchronization, hazards and frames in flight

1. Are execution ordering and memory visibility identical?

No. Execution dependencies order work; memory dependencies also establish the required availability and visibility for relevant accesses.

2. When may a frame context be reused?

After all submissions using its resettable or writable resources have completed under the selected API and queue model.

3. Why is waiting for the whole GPU after each command a poor final design?

It destroys useful overlap, increases latency or idle time and hides the actual dependency structure. Coarse waits remain useful for bring-up and teardown.

4. Context A last used serial 10 and completed serial is 9. Is frame_number modulo two sufficient to reuse A?

No. The context is still in use until its completion condition is satisfied, regardless of index arithmetic.

5. Why can a graphics submission fence be insufficient to prove that a Vulkan presentation-wait semaphore is reusable?

The presentation engine can still be waiting on that semaphore after the graphics submission completes. Use a presentation-aware reuse scheme, such as correctly image-indexed semaphores.

Diagnostic guidance: A flicker that disappears after wait-idle is a useful clue, not a finished fix. Find the specific missing dependency or premature overwrite. A native validation warning should not be dismissed because the equivalent browser example runs.

Lesson 22.4 — Presentation, resizing and device capability policy

1. Why can resizing require more than changing the viewport?

Depth, multisample, history and other attachments can depend on extent and compatibility; projection and presentation resources may also change.

2. What should happen for a zero-height drawable?

Suspend or skip size-dependent rendering and wait for a usable extent while continuing the appropriate event handling.

3. What is the purpose of an explicit feature policy?

It makes required capabilities, optional fallbacks and intentionally unsupported paths explicit and testable.

4. An 800-by-600 logical window has scale factor two in both dimensions. What drawable extent and four-byte depth payload result?

1600-by-1200 and 7,680,000 bytes for the depth payload, excluding implementation overhead.

5. Explain how resetting a fence before an acquire-out-of-date early return can deadlock the next frame.

The fence becomes unsignaled, but the early return issues no submission to signal it. The next wait has no completion event to observe.

Diagnostic guidance: A stretched image suggests aspect ratio or logical/pixel size confusion. A hang after minimizing can indicate a fence reset with no subsequent submission. Persistent ghosts after resize suggest history was not invalidated.

Lesson 23.1 — Radiometry, reflectance and physically based materials

1. What does a BRDF relate?

It relates incoming directional radiance to reflected outgoing radiance density at a surface, with the incident cosine accounted for in the transport integral.

2. What is the Lambertian BRDF for reflectance rho?

rho/pi, in inverse steradians.

3. Why document roughness encoding?

Different models use different mappings, such as alpha=r^2. Without the encoding, the same stored value has different highlight behavior.

4. For rho=0.5 and E=10, compute ideal diffuse outgoing radiance.

5/pi, approximately 1.59155 W/(m^2 sr).

5. Why does a BRDF or normal-distribution value above one not automatically imply energy creation?

These are densities, not total reflected fractions. Energy constraints apply to the appropriately weighted integral, not a pointwise upper bound of one.

Diagnostic guidance: Plastic-looking metals often indicate an unintended diffuse lobe. Highlights that move under object scaling suggest incorrect normal transforms. A roughness slider with an inverted effect may have confused roughness, smoothness and alpha.

Lesson 23.2 — Textures, mipmaps, filtering and tangent-space detail

1. What problem do mipmaps primarily address?

Aliasing under minification by supplying prefiltered lower-resolution representations; they do not automatically solve every sampling problem.

2. What is the approximate total mip-chain storage factor?

Approximately 4/3 of the base texel count for a full square power-of-two chain.

3. Why can mirrored UVs break normal mapping?

They can reverse the tangent frame orientation. A mismatched or omitted handedness sign transforms tangent-space detail incorrectly.

4. How many texels and RGBA8 payload bytes are in the full 256-by-256 mip chain?

87,381 texels and 349,524 bytes, excluding allocation overhead.

5. For n=(0,0,1), t=(1,0,0) and handedness -1, what bitangent is required?

(0,-1,0), because b=-cross(n,t).

Diagnostic guidance: A stable blur is not proof that aliasing was solved correctly. Inspect motion and level selection. A single mirrored seam usually points to tangent conventions; uniformly wrong normal directions can instead indicate channel or color decoding.

Lesson 23.3 — Shadows, light selection and rendering-path tradeoffs

1. Why can excessive shadow bias be visible?

It can detach shadows from their casters or erase contact shadows, trading one artifact for another.

2. Does deferred rendering remove bandwidth cost?

No. It adds or reorganizes attribute storage and traffic; actual performance depends on the implementation and scene.

3. What determines whether clustered lighting helps?

The distribution and count of lights and visible geometry, rejection effectiveness, list-building cost, memory traffic and hardware execution behavior.

4. What is the logical payload of a 1920-by-1080, 16-byte-per-pixel G-buffer write?

33,177,600 bytes, about 31.64 MiB, before other attachments or implementation overhead.

5. A tile light list has capacity 64 but receives 80 lights. What makes overflow handling an engineering requirement rather than an optional optimization?

Out-of-bounds writes are unsafe and silent light loss changes the result. The implementation needs a bounded spill, supported fallback or explicit measured quality policy.

Diagnostic guidance: Floating shadows suggest excessive bias or a depth-sign error. Shadows only on one side suggest light projection or clipping. A faster clustered path with missing lights is a correctness failure until a deliberate quality policy is documented.

Lesson 23.4 — HDR, tone mapping, temporal reconstruction and validation

1. Are tone mapping and correct light transport the same task?

No. Tone mapping transforms values for presentation; it does not repair incorrect transport or lighting units.

2. Why reject history at a camera cut?

The old image generally does not correspond to the newly visible surfaces or sample positions, so reuse can create severe ghosting.

3. Why are still screenshots insufficient for temporal methods?

They hide motion trails, instability, disocclusion failures and history behavior over time.

4. Apply -1 exposure stop and T(x)=x/(1+x) to scene value 4. What value reaches the output transfer encoder?

2/3. Exposure gives 2, then tone mapping gives 2/(1+2).

5. Current sample is 0.2, history 1.0, weight 0.9. What are the blended value and correct camera-cut value?

Blended 0.92; after rejecting history at the cut, 0.2.

Diagnostic guidance: A dark output can result from encoding being skipped; fading trails indicate stale correspondence. A metric improving while thin objects vanish suggests the chosen metric is not sufficient to judge the temporal failure.

Lesson 24.1 — Fixed updates, interpolation and overload policy

1. What fraction remains after two 10 ms steps from 25 ms?

5 ms remains, which is a fraction 0.5 of a 10-ms step.

2. Does a fixed timestep alone guarantee determinism?

No. It controls the integration interval but not all sources of nondeterministic behavior.

3. Why must catch-up work be bounded or otherwise controlled?

Unbounded catch-up can generate more lateness than it removes, starving presentation and input or causing an overload spiral.

4. Under the teaching drop policy, h=10 ms, max steps=4 and accumulated time=105 ms. Give steps, remainder, dropped time and alpha.

Four steps; 5 ms remainder; 60 ms dropped; alpha 0.5.

5. Why can two runs using the same fixed h still diverge? Give two causes other than h.

Examples include different input-to-tick assignment, random-number order, floating-point behavior, unstable container iteration or thread races.

Diagnostic guidance: Objects that speed up at high frame rates may still use callback delta inside fixed rules. A jump applied several times after a stall suggests discrete input was not consumed per tick. Smooth output can hide dropped simulation time; inspect counters.

Lesson 24.2 — Entities, components, handles and data-oriented storage

1. Why separate identity from an object address?

Storage can move or be reused independently of the logical entity lifetime. Identity must not depend on an address remaining fixed.

2. Is handle (7,3) valid for a slot now at generation 4?

No. The generations differ, so it references an expired lifetime.

3. Why defer structural changes during a component iteration?

Immediate insertion or removal can invalidate references and iteration, skip elements or create order-dependent behavior.

4. After swap-removing dense element B and moving C into its slot, which mapping must change?

C's entity-to-dense-index mapping must point to the vacated slot now containing C. B's mapping must be invalidated.

5. Why should a durable scene ID normally be separate from a runtime slot/generation handle?

The runtime handle depends on allocation history and slot reuse. A stable scene identity can be resolved to fresh handles after loading without preserving memory layout.

Diagnostic guidance: A deletion bug that appears only when removing a middle element often indicates an uncorrected reverse mapping. A stale handle that resolves after reuse means the generation comparison or alive check is missing.

Lesson 24.3 — Allocators, task graphs and frame-resource lifetime

1. When is resetting an arena safe?

Only after every CPU and GPU consumer of all old allocations is finished and any required object destruction has occurred.

2. Can task C consume A before A has completed its required output?

No. The required producer output must be complete and visible under the data-sharing model before the consumer proceeds.

3. Why implement a serial graph executor first?

It creates a deterministic correctness reference, validates dependencies and detects cycles before scheduling complexity is added.

4. At offset 17, what range is used by an 8-byte allocation aligned to 16, and how much padding is inserted?

Bytes [32,40), with 15 bytes of padding; next offset 40.

5. For independent A=2 ms, B=5 ms and C=3 ms after both, what are serial time and ideal parallel lower bound?

Serial 10 ms, ideal lower bound 8 ms. Additional overhead can increase actual parallel time.

Diagnostic guidance: Random data corruption after a frame boundary often indicates premature arena reuse. Poor scaling with idle workers can indicate a long critical path, not a broken thread pool. Shared writes without dependency or partitioning are races regardless of the graph's acyclic shape.

Lesson 24.4 — Subsystem boundaries, save formats and reloadability

1. Why is a raw memory pointer unsuitable for a durable save?

The pointer is a process-specific memory location whose lifetime and address do not survive loading into another process.

2. What must a format version enable?

It enables validation of the schema and an explicit migration or rejection policy for supported and unsupported data.

3. Why test partial initialization failure?

It exposes leaks, invalid cleanup order and assumptions that every later subsystem was successfully created.

4. Why does atomic file replacement not automatically prove durability after sudden power loss?

Atomicity describes which file version is visible as an operation; durability depends on flush and filesystem/platform guarantees for data and metadata.

5. Revision 5 completes after revision 6 is active. What should the reload system do and why?

Reject or retire the stale revision-5 result without publishing it. Completion order is not request freshness, and revision 6 must remain active.

Diagnostic guidance: A save that works only in the same process may contain pointers or runtime indices. A failed load that leaves missing entities is not transactional. A newly edited texture reverting later suggests out-of-order reload completion.

Lesson 25.1 — Importing, cooking and content-addressed asset builds

1. Why distinguish source assets from runtime assets?

They serve different authoring and runtime requirements; cooking provides validated, target-specific data and avoids runtime dependence on authoring formats.

2. Must a compression-setting change invalidate a cached build?

Yes, when it affects the artifact or its contract. The new setting must be represented in the key.

3. Why include tool version in a cache key?

A changed converter can change output semantics or bytes for the same source. Versioning makes that dependency explicit.

4. Why is hashing raw concatenation of variable-length input blobs risky without framing?

Different sequences of blobs can have identical concatenated bytes, such as ab+c and a+bc. Use lengths or a structured canonical description.

5. How many bytes does a GEM1 asset with six vertices occupy under the stated eight-f32-per-vertex layout?

200 bytes: 8-byte header plus 6*32 bytes of vertices.

Diagnostic guidance: An asset that looks stale after editing usually indicates a missing dependency or setting in the key. A rebuild on every machine with unchanged semantic inputs may be hashing absolute paths or timestamps. A cache hit with corrupt bytes means existence was mistaken for integrity.

Lesson 25.2 — Streaming, residency and memory budgets

1. Why is compressed file size not the complete memory budget?

Decoded, staging, GPU-resident and old in-flight generations can coexist; compressed bytes represent only one stage.

2. What should rendering do while an optional asset is unavailable?

Use an appropriate stable placeholder or supported lower-detail fallback while respecting the feature policy.

3. Why model loading as explicit states?

They make ownership, completion, failure, cancellation and valid transitions explicit rather than hidden in callbacks.

4. Under the example 100-MiB budget, why is a request ending at 60 MiB resident still rejected?

Its temporary overlap reaches 110 MiB. Admission must account for peak concurrent allocations, not only the eventual steady-state resident total.

5. Does cancelling an already submitted upload make its staging allocation immediately reusable?

No. The consumer may still use it. Wait for the relevant completion before reclaiming it, while suppressing publication of the cancelled result.

Diagnostic guidance: An out-of-memory event despite low resident usage often comes from overlapping decode/staging/retirement allocations. Assets reappearing after scene changes suggest missing request-generation checks. A cancellation crash often indicates storage was freed before a consumer stopped.

Lesson 25.3 — Editors, inspection and undoable commands

1. What is the model/view distinction useful for?

It separates authoritative state and validation from presentation, so multiple views do not maintain conflicting copies of the engine model.

2. What data can make a transform move reversible?

Stable entity identity plus before and after values, with enough revision/context information to apply and reverse the change safely.

3. Why distinguish preview state from committed authoring state?

Preview can be cancelled and may be temporarily invalid. Committed authoring state is what undo, saves and reproducible builds should represent.

4. After undoing two commands, what should a new edit do to an ordinary linear redo stack?

Clear it. The new edit creates a new linear branch whose prior assumptions differ from the old redo commands.

5. Why should a drag preview not normally be written directly into the saved authoring scene before commit?

It would make cancellation and transactional validation difficult and could persist a partial gesture. Keep preview separate, then commit one validated command.

Diagnostic guidance: Undo that works once but not after scene load may use stale runtime handles. A camera moving while typing indicates missing focus routing. An enormous undo stack after one drag suggests previews were committed individually.

Lesson 25.4 — Engine integration and reproducible demonstration

1. What frame interval corresponds to 60 frames per second?

1000/60 ms, approximately 16.667 ms.

2. Why report more than average frame time?

Averages hide stalls, tails, variability and distinct loading or presentation behavior. Preserve raw samples and suitable distribution summaries.

3. What does a clean-machine build test reveal?

Missing dependencies, undocumented environment assumptions, absent generated files, implicit paths and packaging problems.

4. For 99 frames at 10 ms and one at 200 ms, compute the mean and explain why it hides a problem.

Mean=(99*10+200)/100=11.9 ms. The mean hides a 200-ms hitch; max and hitch counts expose it.

5. Why is a clean-directory automated build not sufficient to mark the second-person review complete?

It tests automation in a clean environment, not whether another person can follow the instructions and reproduce the required interactions. Record actual reviewer evidence separately.

Diagnostic guidance: A result that only works in the original working directory usually depends on implicit paths. A smooth average with obvious stutter requires raw frame distributions. A successful build without assets is not a reproducible engine demonstration.

Module 21 review answers

1. Why should the renderer keep an image byte test even when screenshots look right?

Viewers can reinterpret color and hide flips or padding issues. Exact byte and marker tests verify the actual output contract.

2. Can a triangle with every original vertex outside the frustum still have visible area?

Yes. It can cross the view volume; correct polygon clipping may create visible intersection vertices.

3. Which attributes use reciprocal-w interpolation, and what is the depth exception here?

Smooth surface attributes such as UVs use the ratio. Conventional projected z_ndc is already interpolated affinely in screen coordinates.

4. What does a BVH correctness test compare besides execution time?

Hit/miss, nearest primitive identity and t under a stated tie/tolerance policy, using an independent brute-force oracle.

5. Why not promise exact GPU image equality for the CPU reference?

Subpixel precision, coverage, sample positions, color formats and floating behavior can differ; compare under declared conventions and tolerances.

Module 22 review answers

1. Why can correct submission order still be an incomplete native synchronization argument?

The resource may also require memory availability/visibility, a transition or ownership transfer under the API rules.

2. Are WebGPU implicit transitions proof that Vulkan barriers are unnecessary?

No. The managed API supplies different validation and transition behavior; native dependencies still need native expression.

3. What separates frame-context indices from presentation-image indices?

They identify different ownership pools and may have different counts and acquisition order.

4. Why should GPU results include readback instead of only a triangle?

A visual output may not reveal tail bounds or exact compute errors. Numeric readback validates the tested data path.

5. What can a CPU-only run honestly establish for this module?

The ledger, layout arithmetic and other host models; not driver hazard freedom, actual GPU output or GPU performance.

Module 23 review answers

1. Why distinguish radiance from irradiance in a shader notebook?

They have different directional/area units and roles in transport; mixing them can hide missing cosine or normalization factors.

2. Why can a perfect normal-map seam on one mesh be insufficient?

Mirrored UVs, degenerate parameterization and nonuniform scaling expose different tangent-frame failures.

3. Why is a lower light-list count not a measured speedup?

List construction, lookup, memory and GPU execution overhead also contribute; measure the equal-quality workload.

4. Does an HDR intermediate prove HDR display output?

No. The engine may tone-map the intermediate to an SDR surface; display negotiation and color handling are separate.

5. Why preserve camera-cut and disocclusion failures in the report?

They demonstrate the limits and behavior of correspondence/rejection rather than concealing temporal artifacts behind still images.

Module 24 review answers

1. What does dropping accumulated ticks sacrifice?

Simulation no longer tracks all elapsed real time; this must be compatible with the product and recorded explicitly.

2. Why is a valid handle lookup not permission to hold a component pointer forever?

Subsequent structural changes can move or destroy storage even while logical identity remains meaningful.

3. Can an acyclic task graph still race?

Yes. Missing edges or undeclared overlapping writes permit unsafe simultaneous access.

4. What makes a load transactional?

It constructs and validates a full candidate before publication, leaving the old world unchanged if any step fails.

5. What protects against an old reload overwriting a new one?

Request/revision tokens checked at completion and publication, plus safe retirement of discarded candidates.

Module 25 review answers

1. Which key inputs prevent a stale content build after a tool upgrade?

Tool/importer version, target format, semantic settings, source digest and dependency digests.

2. Why can bounded worker count still exceed the memory budget?

Each worker or queue stage can retain multiple large decoded/staging allocations; bound bytes and overlap, not just threads.

3. What must happen to editor history when its target scene revision is replaced?

Clear it or apply a deliberately revision-aware remapping policy; never replay stale commands blindly.

4. What does the software-only Gate E allowance exclude?

Untested GPU support, hazard-freedom and timing claims. CPU integration, ownership, cooking, inspection and other mandatory evidence still apply.

5. What must a reviewer reproduce besides opening a screenshot?

A clean build/run, a boundary test, a live edit and save/reload under the README, followed by the learner explaining dependencies and tradeoffs.